

SCALABILITY & PRODUCTION READINESS SPRINT

AirOps - Frontend + Data Performance Report

Overall answer: YES WITH ARCHITECTURAL CHANGES. The interface and data-access pattern handle a 520,000-flight dataset, but the current single API process and unoptimized database path are not reliable under concentrated concurrent traffic.

Member: Ibrahim - Frontend + Data Performance Lead

Project: AirOps airline operations scalability lab

Verified dataset: 520,000 flights; 2,138,472 total rows across five tables

Audit date: September 3, 2026

Evidence: Repository audit, live API tests, PostgreSQL plans, UI checks and a local break probe

Build -> Measure -> Break -> Find the bottleneck -> Improve -> Measure again

Executive scalability assessment

Decision: The project proves that a bounded frontend can work with a very large dataset. It does not yet prove production capacity at 50-1,000 concurrent users. In the local break test, database pool pressure and expensive list/search queries caused 503 responses and ~10-second tail latency.

Question	Evidence-based answer
What was tested?	520k data volume, live UI, API timing, payload size, SQL plans, 10/50/100/150-client local read probe, API stop/restart.
What worked?	Pagination keeps the DOM bounded; dashboard/list/detail pages work; current payload is small; errors are visible; service recovered after load and restart.
What broke?	At 50+ concentrated clients, requests queued behind the 20-connection pool and many returned HTTP 503 after connection timeouts.
Biggest bottleneck	PostgreSQL work per request: full scans/sorts, wildcard ILIKE search, exact COUNT on each page, on-demand dashboard aggregates and limited pool capacity.
What was improved?	Frontend caching/deduplication, stable keys, 300 ms search debounce, session airport cache, smaller list projection, memoization and cache invalidation.
Current comfortable capacity	Not certified. A 10-client probe had 0% errors but p95 was ~5.0 s because search was slow; 50 clients were unstable in this environment.
Next scaling step	Fix the load scripts, add evidence-led indexes and query changes, reduce count/aggregate work, add backpressure/monitoring, then rerun 50/100/150 before attempting 500/1,000.

Capacity answer

Load	Current status	What it means
10	Degraded	0% errors, but p95 5.01 s in the short local mixed read probe.
50	Unstable	p95 10.08 s; 50.0% failures/timeouts in the same probe.
100	Unstable	p95 10.38 s; 92.35% failures.
150	Unstable	p95 10.66 s; 91.58% failures.
500	Not run	Requires query/index work, cached aggregates, pooler, load shedding and horizontally scaled API.
1,000	Not run	Requires the 500-user changes plus production-scale validation, autoscaling, read scaling and stronger observability.

Important: these are local, 10-second read-only probe results with 750 ms think time. They expose a real failure mode but are not a replacement for repeatable k6 tests on production-like infrastructure.

1. Project introduction

AirOps is a deliberately small but production-style airline operations application. The goal is not feature volume; the goal is to watch user experience and system reliability as the dataset and request concurrency increase.

Ibrahim's part of the sprint asks a practical question: what does the user see when the backend is slow, overloaded or unavailable? The work therefore covers the large-data interface, API behavior from the browser, failure states, request reduction and evidence for where the architecture stops being reliable.

1.1 Architecture used

Layer	Technology	Purpose
Frontend	Next.js 14, React 18, TypeScript, Tailwind	Dashboard, bounded table, filters, detail/update flow, diagnostics and failure UX.
Client data	TanStack Query	Caching, in-flight deduplication, background refresh, retries and invalidation.
API	Node.js + Fastify	Validated REST endpoints, structured logs, CORS, timing hooks and graceful shutdown.
Database	PostgreSQL 16 + Drizzle	Relational flight/airport/aircraft/event/incident data; pool max is 20.
Test tools	Benchmark script, EXPLAIN ANALYZE, load scripts	Single-user baseline, query evidence and planned concurrent-user scenarios.

1.2 Verified large dataset

Table	Verified rows	Notes
flights	520,000	Primary operations dataset.
flight_events	1,602,332	Historical status/activity records.
incidents	15,850	Operational incident records.
airports	40	Static reference data.
aircraft	250	Fleet reference data.
Total	2,138,472	All five relational tables.

Documentation mismatch: The seed code currently inserts 2,000 flights per batch. The older data-model document says 5,000. The report uses the code as the source of truth.

2. What I built

2.1 High-volume frontend

- Operations dashboard with eight KPI cards, status distribution and manual/background refresh.
- Flight table with server-side pagination (20 by default, API maximum 100), search, status/origin/destination filters and sorting.
- Flight detail view with airport, aircraft, timings, events, incidents and a status-update workflow.
- Diagnostics page showing API reachability, database state, query latency and process uptime.
- The browser never tries to render 520,000 rows. Only the current page is fetched and rendered.

2.2 API and data foundation

- GET /health, /api/airports, /api/flights, /api/flights/:id and /api/dashboard, plus PATCH /api/flights/:id/status.
- Zod validation, centralized JSON errors, Pino request logging, request-duration logging, CORS and graceful shutdown.
- Five-table Drizzle schema with foreign keys and cascading event/incident cleanup.
- Bulk seeder designed for 10k, 100k, 500k and 1m flights, with child events/incidents generated in chunks.
- Status update and event insertion are wrapped in one database transaction, which protects against partial writes.

2.3 Frontend performance and resilience work

- TanStack Query cache and in-flight request deduplication.
- Stable primitive query keys so a real filter/page change triggers one new request.
- 300 ms search debounce to avoid one request per keystroke.
- Airports cached for the browser session (staleTime and gcTime set to Infinity on the page query).
- Flight-list response projected to only fields rendered by the table.
- Memoized metric cards and flight rows; targeted cache invalidation after a status change.
- Loading skeletons, retry actions, empty states, timeout/cancellation and development-only failure injection.

3. Screenshots from the live 520k dataset

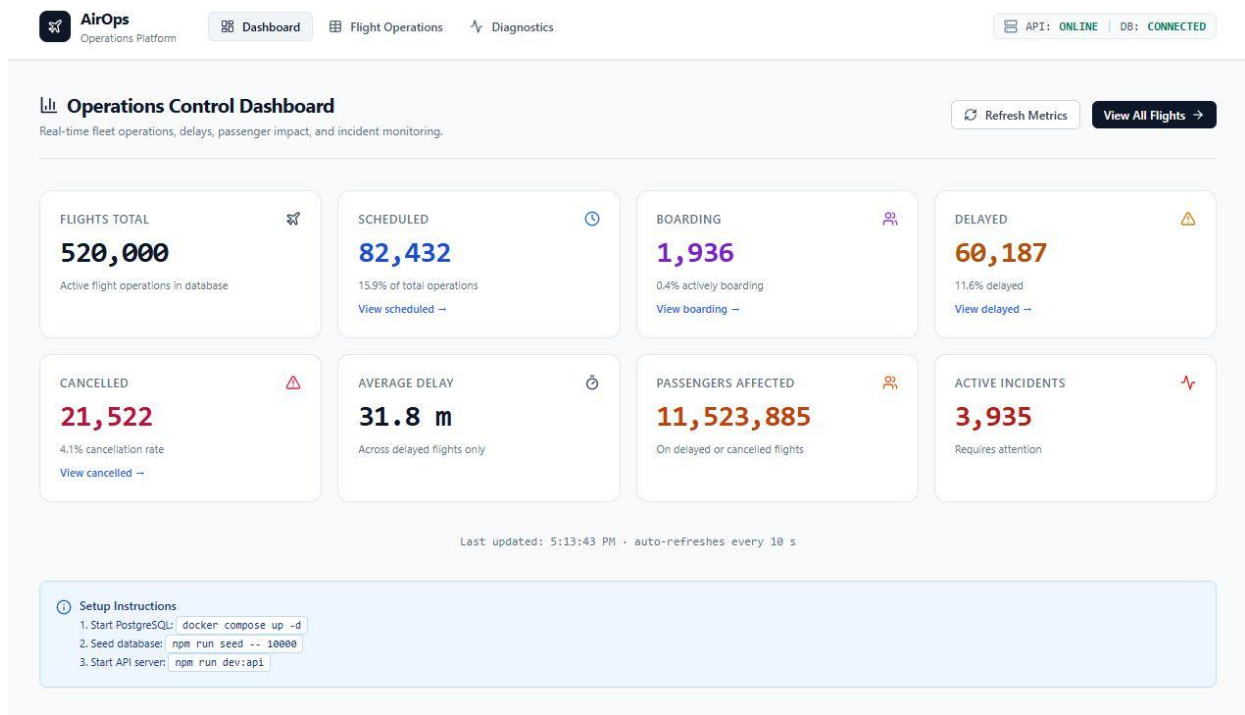


Figure 1 - Dashboard reading real metrics from 520,000 flights.

The dashboard uses one compact API response (0.3 KB in the benchmark), but the database still recomputes full-table aggregates on each refresh.



Ibrahim | September 3, 2026 | Page 6

4. What I tested

Check	Result	Evidence
API TypeScript	PASS	tsc --noEmit completed.
Frontend TypeScript	PASS	tsc --noEmit completed.
API production build	PASS	tsup built dist/server.js in 82 ms.
Frontend production build	PASS	Next.js generated all routes successfully.
Live database	PASS	Health endpoint connected; 520,000 flights returned by pagination.
Single-user API baseline	PASS	10 warm-run samples per endpoint.
SQL execution plans	PASS	EXPLAIN ANALYZE captured on the live 520k database.
Concurrent break probe	PARTIAL	10/50/100/150 read-only clients; short local run, not formal k6.
Service restart	PASS	UI showed offline state and returned online after restart.
k6 suite	BLOCKED	Saved smoke run made 0 requests because request configuration is invalid.

4.1 Production frontend output

Route	Route JS	First-load JS
/dashboard	0.18 KB	116 KB
/flights	4.32 KB	108 KB
/flights/[id]	3.62 KB	116 KB
/diagnostics	3.57 KB	90.8 KB
Shared	-	87.2 KB

These build sizes confirm route-level splitting is working. They do not replace real browser FCP/LCP/INP measurements, which are still missing.

5. Actual 520k-record API baseline

The following results were measured locally on September 3, 2026 after three warm-up requests. Each endpoint was sampled 10 times. These are current-state timings, not before/after optimization results.

Endpoint	Avg	P95	Payload	Result
GET /health	2.1 ms	3.2 ms	0.1 KB	Fast
GET /api/airports	2.5 ms	2.9 ms	4.6 KB	Fast; cached in browser
GET /api/dashboard	98.8 ms	109.5 ms	0.3 KB	Good alone; full scan
Flights: 20 rows	273.6 ms	290.1 ms	6.7 KB	Usable alone; full sort
Flights: 50 rows	346.7 ms	450.8 ms	16.6 KB	Higher payload + same scan
Page 50 / 20 rows	290.0 ms	332.4 ms	6.7 KB	Offset cost starting
Status = DELAYED	126.6 ms	139.2 ms	6.6 KB	Sequential status scan
Search = AA	951.9 ms	990.7 ms	6.7 KB	Slowest read path
Flight detail	86.7 ms	151.9 ms	0.9 KB	PK lookup is efficient

User-experience meaning: The table feels reasonable for a single user, but a search already consumes about one second before frontend rendering. Under concurrency, these expensive searches occupy pool connections and amplify queueing.

5.1 Payload result

The current 20-row flight-list response is 6,826 bytes (6.7 KB). This is better than the older documentation's 10-22 KB estimates. A true before/after percentage cannot be claimed because the pre-projection response was not saved or rerun.

6. PostgreSQL bottleneck evidence

EXPLAIN ANALYZE confirms that the main performance cost sits below the React interface. The database repeatedly scans and sorts the 520,000-row flight table.

Query	Plan evidence	Execution
Dashboard aggregates	Parallel sequential scan of all flights	101.7 ms
Exact pagination COUNT	Parallel sequential scan of all flights	42.6 ms
Page 1 sorted list	Full parallel scan + 3 joins + top-N sort	292.5 ms
Deep offset (980)	Full scan/sort and discard earlier rows	244.7 ms
ILIKE %AA% search	Full scan + airport joins; 158,920 rows/worker rejected	429.9 ms
Status count	Parallel sequential scan; no status index	47.1 ms
Detail by ID	Primary-key index + small joins	3.0 ms

Why this becomes a cliff

- Every list request performs two pieces of work: an exact COUNT and the data query.
- Sorting by scheduled departure has no supporting index, so returning 20 rows can still scan/sort the full table.
- Leading-wildcard ILIKE search cannot use a normal B-tree index and was the slowest endpoint.
- Dashboard refreshes perform live aggregates instead of reading cached or precomputed metrics.
- The API pool allows 20 database connections. Expensive concurrent requests fill it; later requests wait and time out after 5 seconds.

7. Break-it test: concentrated mixed reads

I ran a short local read-only probe against the 520k database using 60% paginated flight reads, 20% wildcard searches, 10% dashboard requests and 10% health checks. Each client waited 750 ms between requests. Requests had a 10-second timeout.

Clients	Requests	P50	P95	P99	Failures
10	42	1.22 s	5.01 s	7.93 s	0.00%
50	92	7.51 s	10.08 s	10.08 s	50.00%
100	170	6.77 s	10.38 s	10.40 s	92.35%
150	273	5.81 s	10.66 s	10.67 s	91.58%

What actually broke: API logs repeatedly reported 'timeout exceeded when trying to connect'. The server returned HTTP 503 after roughly 5.25 seconds while the database pool was saturated. Search failed first and most consistently.

Recovery

After the probe stopped, the API returned 200 responses again without being restarted. The health payload reported the database connected with a 1.56 ms ping. This shows the failure was capacity/queue pressure rather than permanent corruption or a process crash.

Limits of this result

- It is a local probe on one machine, not a production-sized distributed load test.
- The run is short, so it does not prove memory stability or sustained capacity.
- It is read-only and does not prove write safety or lock behavior.
- It is still useful because it reproduced a specific, logged failure mechanism and recovery path.

8. Failure handling seen by the user

I stopped the API while the production frontend remained open. Within the diagnostics polling cycle, the interface changed to API OFFLINE / DB DISCONNECTED, showed 'Failed to fetch', kept navigation working and exposed a Check Now action.

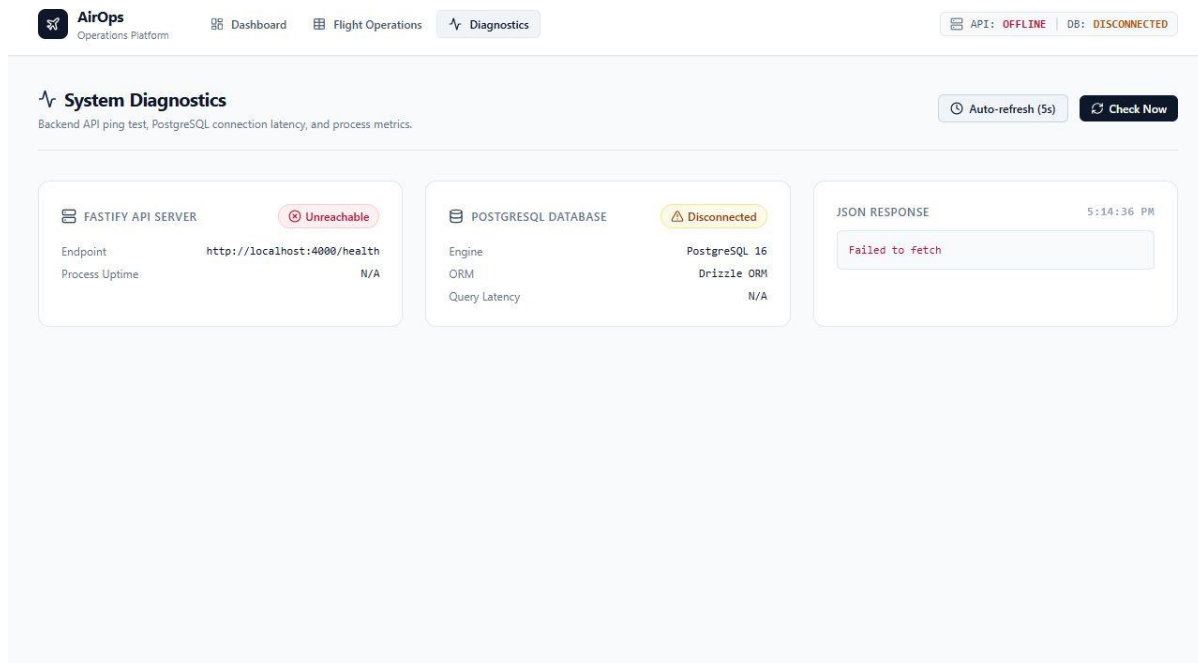


Figure 3 - API stopped: clear offline state instead of a broken page.

I then restarted the API. The next polling cycle changed the page back to ONLINE / CONNECTED and displayed fresh uptime and database latency without reloading the frontend.

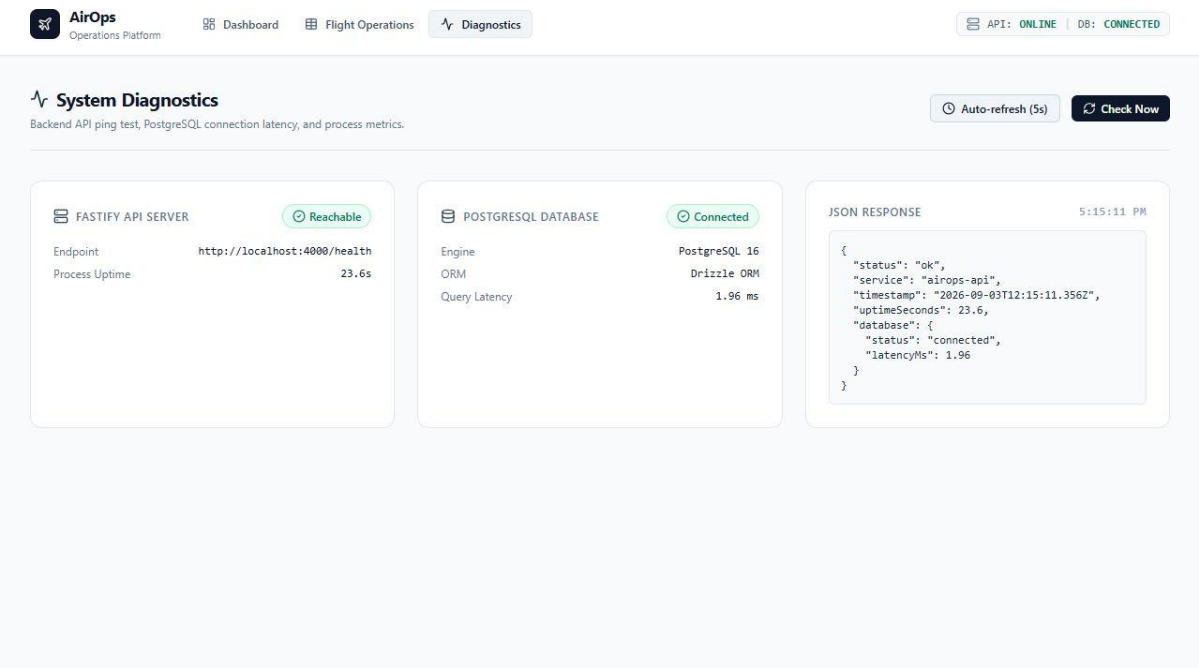


Figure 4 - API restarted: automatic recovery detected by the UI.

8.1 Implemented failure behaviors

Condition	Current behavior	Evidence status
Loading / slow response	Skeleton rows/cards and loading indicators.	Implemented in shared components and pages.
HTTP 500 / 503	Typed error message, warning state and retry action.	Implemented; 503 reproduced during break probe.
Network/API down	Offline diagnostics and network-specific error text.	Live stop/restart verified.
Timeout	AbortController with 5 s dashboard and 10 s list/detail timeouts.	Implemented; connection timeout observed server-side.
Retry	TanStack Query retries GET failures once; mutations do not retry.	Code verified. Older docs incorrectly say 3 retries.
Empty results	Contextual empty state and filter-reset action.	Implemented; not re-measured under load.
Stale response	Query cancellation and stable keys reduce stale overwrites.	Code verified.
Injected failures	Development-only latency/error/timeout wrapper.	Implemented; production build intentionally disables it.

Data safety under concurrent writes

The PATCH route writes the new status and its flight event inside one transaction, so those two operations succeed or fail together. However, the row has no version field or optimistic-lock check. Two writers can overwrite each other (last write wins) while both events remain. The supplied write-contention scenario has not produced valid recorded results, so data safety under competing updates is not yet proven.

9. Optimizations completed

Change	Why it helps	What is proven
Server-side pagination	Bounds network and DOM work.	20 rows displayed from 520k; max API page is 100.
300 ms search debounce	Avoids a request for every keypress.	Code behavior is clear; no saved browser trace.
TanStack Query	Deduplicates/caches GET requests.	Configuration and page usage verified.
Airports session cache	Stops repeated static-data fetches.	staleTime/gcTime Infinity verified.
Stable query keys	Only meaningful state changes refetch.	Primitive key construction verified.
List projection	Cuts unused JSON fields.	Current 20-row payload measured at 6.7 KB.
Memoized rows/cards	Reduces repeat React work.	Implementation verified; CPU saving not measured.
Targeted invalidation	Keeps detail/list data fresh after write.	Mutation invalidation verified.

9.1 Before/after evidence status

Honest result: A numeric before/after performance claim is still incomplete. The repository contains current measurements and several estimates, but it does not contain a saved pre-optimization build, raw DevTools trace or valid k6 output for comparison.

Metric	Current / defensible statement	Do not claim yet
Search calls	Debounce turns a fast 5-keystroke input into about 1 request after idle.	Measured 80% network reduction without a trace.
Airport calls	Page query caches airport data for the session.	Measured 5-to-1 reduction without recorded navigation.
Flight payload	Current 20-row response is 6.7 KB.	50% reduction unless old payload is reproduced.
Page speed	Production build sizes are known.	25-50% faster FCP/CPU; prior values are estimates.
Capacity	The local probe shows a cliff at 50 concentrated clients.	Stable at 50 based on the existing documentation.

10. k6 framework: built, but not yet valid

The repository contains smoke, mixed-load, progressive-stress, spike and write-contention scenarios with presets from 1 to 1,000 VUs. That is useful scaffolding, but it is not the same as executed evidence.

Issue	Impact	Required fix
responseType: 'json' in HTTP options	k6 rejects it; saved smoke run sent 0 requests.	Remove it or use a supported response type; parse JSON from the response.
options references options.vus while building itself	load.js and write-contention.js can fail during module evaluation.	Calculate targetVUs before exporting options.
No raw result files	50/100/150/250 claims cannot be verified.	Export JSON/CSV and retain console summaries for every tier.
Threshold rate >100 req/s	May conflict with deliberate user think time and create false failures.	Set arrival-rate goals from a workload model or remove the arbitrary threshold.
k6 unavailable in this environment	Formal suite could not be rerun here.	Install/restore k6, then run smoke before higher tiers.

The existing smoke-test-output.txt confirms the first issue: setup ended with zero HTTP requests and 'json does not belong to ResponseType values'. Therefore the older 50/100/250 stability table must be treated as expected/example data, not measured performance.

Correct validation order

1. Fix the k6 request options and self-referencing VU configuration.
2. Run smoke and confirm non-zero requests and 100% expected checks.
3. Run 10, 50, 100 and 150 VUs on the same build/data/hardware; repeat each important level at least three times.
4. Capture API logs, PostgreSQL active connections, pool wait/timeout counts, CPU, memory, disk I/O and event-loop lag during the same window.
5. Apply one optimization batch, then rerun the identical suite for a real before/after comparison.
6. Only after 150 is repeatably stable should 500 and 1,000-user tests be attempted.

11. What must change to scale

First: make 50-150 users reliable

- Add indexes based on the captured plans: `scheduled_departure` for sort/pagination, selective `status/date/airport` combinations for real access patterns, and `pg_trgm` or a better search path for leading-wildcard search.
- Remove unnecessary exact COUNT work from every page where possible; use cached/approximate totals or return `hasNext` instead.
- Move dashboard aggregates to a short-TTL cache, rollup table or materialized view instead of rescanning 520k flights every 10 seconds per client.
- Use `keyset/cursor` pagination for deep navigation instead of `OFFSET`.
- Add request concurrency limits, backpressure and sensible 429/503 responses before the pool is exhausted.
- Tune the pool with PostgreSQL limits and measured query duration; add PgBouncer when multiple API processes share the database.
- Fix k6 and define an SLO such as `p95 < 1 s` and `errors < 1%` for the critical list/detail paths.

At about 500 concurrent users

- Run multiple stateless Fastify instances behind a load balancer.
- Use PgBouncer/connection governance so API scale-out does not create a database connection storm.
- Cache hot reference data and dashboard/read-heavy responses; invalidate deliberately after writes.
- Add observability for endpoint `p95/p99`, errors, pool wait time, slow queries, CPU/memory and recovery time.
- Protect writes with optimistic concurrency/version checks and idempotency where repeated client actions are possible.

At about 1,000 concurrent users

- Keep horizontally scaled API instances with autoscaling based on latency/CPU/pool pressure, not request count alone.
- Serve static frontend assets through CDN/edge caching.
- Separate read/reporting pressure from transactional writes; introduce read replicas only after query/index tuning.
- Use queues for genuinely asynchronous work, not for normal read requests.

- Consider partitioning or specialized search/analytics only if measured plans show that simpler optimizations are no longer enough.
- Run sustained, spike, slow-database, restart and write-contention release gates on production-like infrastructure.

12. Final answer: CAN WE SCALE?

Verdict: YES WITH ARCHITECTURAL CHANGES - and NOT YET a clean production-capacity YES for the current baseline.

What the sprint has proved

- A well-bounded interface can browse and operate on a 520,000-flight / 2.14-million-row relational dataset.
- Frontend request amplification and payload bloat have been reduced in the implementation.
- The UI stays understandable when the API is stopped and automatically recognizes recovery.
- The main bottleneck is measurable and explainable: expensive database work fills a 20-connection pool, causing queueing and 503 timeouts.

What is still required for a clean YES

- A working k6 suite with saved, repeatable results at 50, 100 and 150 users.
- A true pre-change benchmark and matching post-change benchmark.
- Indexes/query changes/cached aggregates proven with EXPLAIN ANALYZE and load results.
- Write-contention evidence proving no lost updates or inconsistent history.
- Monitoring evidence for failure detection and measured recovery time.
- Only then: controlled 500 and 1,000-user tests on production-like infrastructure.

Vision71 can take substantially larger applications if this method becomes the standard: measure honestly, expose the bottleneck, fix the smallest effective layer, and repeat the same test. The current project shows the right direction, but the final capacity claim must be earned with repeatable evidence.

Evidence reviewed

Repository source and Markdown documentation; live API responses; apps/api/scripts/benchmark.mjs; apps/api/scripts/query-plans.mjs; load-tests/.js; smoke-test-output.txt; production builds; and browser screenshots captured from the running application.*